TÉCNICO SOLAR BOAT

Técnico Solar Boat

Universidade de Lisboa - Instituto Superior Técnico

Report Template

Last Change: CHANGE

Index

1 Buoyancy **1**
1.1 simplificação 1
1.2 Adaptação à morfologia da mesh 1
1.3 Cálculo simplificado do Coeficiente de Buoyancy 2

2 Simplificação Fossen **2**
2.1 Matriz massa e inércia 2
2.2 Damping 3
2.3 Forças de Restauro 3
2.4 Forças resultantes 3
2.5 Integração das acelerações 3
2.6 Inputs 4

3 Thrusters **4**

4 Sensores **5**
4.1 GPS 5
4.2 ARHS 5
4.3 CÂMERA 5
4.4 LIDAR 5

5 Conectar ao ROS2 via rosbridge **6**
5.1 Linux 6
5.2 Windows- Unreal Engine 6

1 Buoyancy

1.1 simplificação

Para calcular a buoyancy decidi utilizar um modelo baseado em 4 pontos (dois à frente e dois atrás) ao invés de utilizar as ferramentas do unreal engine visto que não nos foram dadas informações específicas de volume deslocado e para evitar o conflito de forças.

Assim, para calcular a buoyancy, utilizei a simplificação de uma malha consituida por 4 pontos, estes pontos correspondem a 4 sockets (pontos invisiveis fixos na mesh).

- FL (Front-Left): Frente Esquerda
- FR (Front-Right): Frente Direita
- BL (Back-Left): Trás Esquerda
- BR (Back-Right): Trás Direita

Este pontos irão gerar três graus de liberdade - heave, pitch and roll.

Dentro da função tick (a cada frame), é calculada a distância entre o socket e a superfície a àgua(no eixo z)

- É utilizada a função getsocketlocation()que devolve as coordenadas 3D da posição do socket em relação ao mundo combinada com o nível de refrência da água.

Quanto o ponto se encontra submerso aplica-se a lei de Hooke:

$$F_{z,i} = \begin{cases} k \cdot (Z_{surf} - Z_i), & \text{se } Z_i < Z_{surf} \\ 0, & \text{caso contrário} \end{cases} \tag{1}$$

A aplicação da lei de Hook faz obter 4 froças verticais.

- Heave- corresponde à força vertical total

$$F_{Heave} = \sum_{i=1}^{4} F_{z,i} \tag{2}$$

- Roll e pitch- correspondem ao somatório do momento calculado através da distância dos sockets ao centro de massa do barco

$$\vec{\tau}_{total} = \sum_{i=1}^{4} (\vec{r}_i \times \vec{F}_{z,i}) \tag{3}$$

(para calcular a distancia dos sockets ao centro de gravidade são primeiro caculadas as coordenadas dos sockets em relação ao pivot(RTS-actor) com a função getsocketTransform seguida de getlocation e as coordenadas do centro de massa eem relacao ao pivot através do comando getBodyInstance()segiodo getMassSpaceLocal()seguido de getLocation())

1.2 Adaptação à morfologia da mesh

Como o OTTER é um catamaran (dois cascos laterais e convés suspenso no meio), a simplificação para os 4 pontos nas extremidades dos pontões encaixa perfeitamente. A utilizacão de um sistema de flutuabilidade basedo volume por simplificação do formato pode igorar o espaço vazio debaixo do convés conduzindo a um movimento irrealista do barco. Fixar sockets nos pontões garante que essa àrea não desloca agua. Utilizar mais pontos fixos nos pontões seria mais rigoroso mas os 4 pontos utilizados (o mais afastados possível para garantir a estabilidade-momentos de restauro maiores) são o mínimo suficiente.

1.3 Cálculo simplificado do Coeficiente de Buoyancy

Para evitar usar um coeficiente inventado por necessidade paramétrica, o coeficiente de buoyancy pode ser calculado a partir dos parâmetros físicos fornecidos.

O volume total pode ser calculado dividindo a massa do barco pela densidade do veículo :

$$V_{total} = \frac{Massa}{\rho_{veiculo}} \tag{4}$$

Assumindo a altura total do casco, a área de flutuação na linha de água é calculada por:

$$A_w = \frac{V_{total}}{H_{casco}} \tag{5}$$

Aplicando o modelo linear de forças de restauro hidrostáticas de Fossen, a constante de restauro total no eixo Z é o produto da Densidade do Fluido , aceleração da Gravidade (g) e a área calculada:

$$Z_{restauro} = \rho_{fluido} \cdot g \cdot A_w \tag{6}$$

Como o modelo de pontos utiliza 4 sockets, constante de restauro total é dividida de forma igual pelos pontos para obter o coeficiente de buoyancy de cada um:

$$k = \frac{Z_{restauro}}{4} \tag{7}$$

2 Simplificação Fossen

O modelo de Fossen baseia-se na seguinte equação não linear de movimento:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \boldsymbol{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \boldsymbol{g}(\boldsymbol{\eta}) = \boldsymbol{\tau} \tag{8}$$

A simplificação consistiu em transformar este sistema acopulado em equações lineares independentes, além de ignorar por completo a matriz de Coriolis (forças que afetam trajetórias a altas velocidades), sendo utilizada apenas:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \boldsymbol{g}(\boldsymbol{\eta}) = \boldsymbol{\tau} \tag{9}$$

2.1 Matriz massa e inércia

No modelo de fossen a matriz de inércia e massa total do veículo correspondem à soma da matriz de soma do corpo rígido com a matriz de massa adicionada.

Numa forma não simplificada, a matriz possui 36 elementos(6x6) não nulos. Para a implementação teórica no unreal e de acordo com a tabela de parâmetros fornecida a matriz foi simplificada a uma matriz diagonal.

$$M_{simplificada} = \begin{bmatrix} m + X_{\dot{u}} & 0 & 0 & 0 & 0 & 0 \\ 0 & m + Y_{\dot{v}} & 0 & 0 & 0 & 0 \\ 0 & 0 & m + Z_{\dot{w}} & 0 & 0 & 0 \\ 0 & 0 & 0 & I_{xx} + K_{\dot{p}} & 0 & 0 \\ 0 & 0 & 0 & 0 & I_{yy} + M_{\dot{q}} & 0 \\ 0 & 0 & 0 & 0 & 0 & I_{zz} + N_{\dot{r}} \end{bmatrix} \tag{10}$$

2.2 Damping

No modelo de fossen as forças dissipativas (atrito com a água) são representadas por uma matriz 6x6. Neste trabalho, tal como a matriz da massa, também simplificada a uma matriz diagonal sendo apenas utilizada a componente damping linear.

$$D_{simplificada} = \begin{bmatrix} D_x & 0 & 0 & 0 & 0 & 0 \\ 0 & D_y & 0 & 0 & 0 & 0 \\ 0 & 0 & D_z & 0 & 0 & 0 \\ 0 & 0 & 0 & D_\phi & 0 & 0 \\ 0 & 0 & 0 & 0 & D_\theta & 0 \\ 0 & 0 & 0 & 0 & 0 & D_\psi \end{bmatrix} \tag{11}$$

2.3 Forças de Restauro

As forças e momentos de restauro hidroestáticos (resultantes da interação gravidade/buoyancy) também foram simplificados para o eixo heave e rotações de roll e pitch sendo 0 nos outros eixos.

$$g(\eta) = \begin{bmatrix} 0 \\ 0 \\ Z_{restauro}(z) \\ K_{restauro}(\phi) \\ M_{restauro}(\theta) \\ 0 \end{bmatrix} \tag{12}$$

2.4 Forças resultantes

Isolando o termo ($M\dot{\nu}$) as forças resultantes equivalem à propulsão (τ) menos as forças de atrito ($D(\nu)\nu$) mais as de restauro ($g(\eta)$), obtendo a seguinte expressão:

$$F_{resultante} = M\dot{\nu} = \tau - D(\nu)\nu + g(\eta) \tag{13}$$

Pelo facto da diagonalização das matrizes (desacoplamento dos eixos), a resolução matricial deste sistema consiste em 6 equações escalares independentes.

Considerando os graus de liberdade de translação e rotação no plano horizontal (Surge, Sway e Yaw), onde as forças de restauro são nulas, a força resultante é apenas a propulsão menos o atrito. Por exemplo, para o eixo X (Surge):

$$F_x = \tau_x - (D_x \cdot u) \tag{14}$$

Considerando os graus de liberdade(eixos) influenciados pela flutuabilidade (Heave, Roll e Pitch), o termo $g(\eta)$ é introduzido. Por exemplo para o eixo Z (Heave):

$$F_z = Z_{restauro} - (D_z \cdot w) \tag{15}$$

Depois de obtidas as forças resultantes em cada eixo, as acelerações($\dot{\nu}$) são calculadas pela divisão forças/ respetivas massas.

2.5 Integração das acelerações

Após calculadas as acelerações basta integrá-las de modo a obter as velocidades que se materializam no movimento da Otter a cada *frame*, na simulação. Para isso, é utilizado

um método de integração - Método de Euler (a nova velociade correrponde à soma da velocidade atual com o produto da aceleração pelo intervalo de tempo decorrido):

$$\nu(t + \Delta t) = \nu(t) + \dot{\nu}(t) \cdot \Delta t \tag{16}$$

Todos os cálculos relativos ao modelo fossen estão inseridos dentro da função tick que é executada pelo Unreal multiplas vezes por segundo (Ex. 60FPS-60x por segundo).

Esta função disponibiliza O `DeltaTime` que corresponde ao tempo que decorreu desde a última atualização (pode variar entre frames) e é precisamente este o valor utilizado no método de Euler realizado para os 6 graus de liberdade. Este ciclo de atualização é tão rápido que cria a ilusão do movimento ser contínuo quando na verdade são somadas velocidades.

2.6 Inputs

Para controlar o otter pelo teclado usei o Enhanced Input System que através de modifiers transforma as teclas em inputs. Basicamente configurei uma input action 2D para poder controlar simultaneamente dois graus de liberdade(x e y). Por default cada tecla envia o valor 1.0 para o eixo x, assim no Input Mapping Context, através dos modifiers que já referi alteerei isso mesmo. Usei para o W e S o swizzle Input Axis Value que redireciona o input para o eixo Y, posteriormente adicionei o modifier negate às teclas S e A para gerar valores negativos. É importante relembrar que no UE o Y corresponde à frente e o X ao lado, portanto O W E S controlam o avanco/recuo e o A e D o torque em Z. Para finalizar no blueprint graph adicionei o modulo dos enhanced inputs e separei os pinos em X e Y. Liguei o X ao argumento lado e o Y ao argumento frente da função que atribui a força aos motores pelo motivo que acabei de referir.

3 Thrusters

De acordo com os dados da blueRobotics relativamente aos Thrusters T200 a curva de desempenho (corrente-força) é a seguinte:

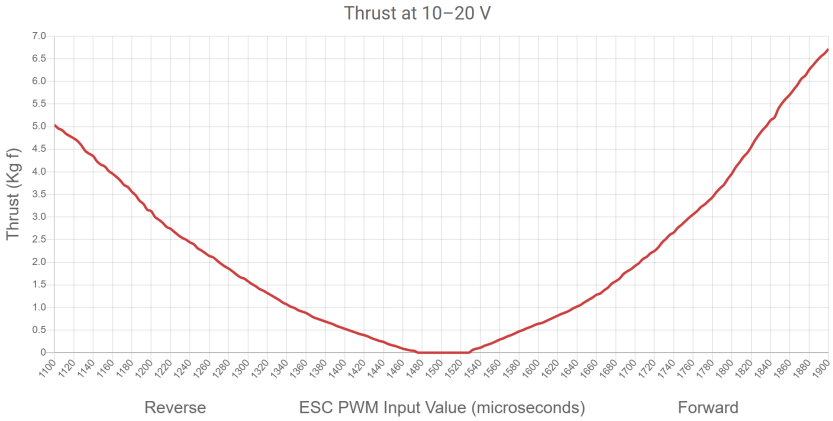

Figure 1: Caption

Para traduzir a curva numa função para utilizar no código c++ do Unreal foi utilizada uma regressão polinomial de 2º grau. A assimetria das pas obriga a 2 equações distintas.

A curva da direita corresponde ao avanço - corrente positiva. Foward:

$$T_{kgf} = 0.012C^2 + 0.15C \quad \text{para} \quad C > 0 \tag{17}$$

A curva da esquerda corresponde ao recuo - corrente negativa (A hidrodinâmica da hélice é menos eficiente no recuo) Reversse:

$$T_{kgf} = -\left(0.01\,|C|^2 + 0.12\,|C|\right) \quad \text{para} \quad C < 0 \tag{18}$$

4 Sensores

4.1 GPS

Para a implementação dO GPS no código C++ recorre-se à função getsocketlocation() do Unreal Engine para extrair as coordenadas globais dos sockets GPS. Em seguida é adicionado o ruído com a intenção de replicar as imprecisões de um GPS real. Utiliza-se a função FMath::VRand() para gerar um vetor 3D de direção aleatoria (comprimento 1cm) que é multiplicado por uma distância sorteada pela função FMath::RandRange() (um numero aleatorio entre 0 e a variavel de variacao de ruído).

4.2 ARHS

Para a implementação do AHRS no código C++ recorre-se à função getSocketrotation() do Unreal Engine para extrair a orientação angular nos 3 eixos (pitch, yaw, roll) do socket de AHRS. Em seguida é adicionado o ruído para cada eixo . É utilizada a função FMath::RandRange() para gerar um valor aleatório de acordo com a variável variância estipulada. É importante salientar que a abordagem deste sensor para a simulação é bastante simplificada, ao invés de dados brutos provinientes do giroscópio ou acelerómetro fornece os ângulos finais não sendo necessário filtros de fusão sensorial.

4.3 CÂMERA

Para implementar a câmera basta recorrer às ferramentas nativas do Unreal Engine -UcameraComponent que realiza internamente a captaçao e renderização de imagem. Através da função SetupAttachment() este sensor foi acoplado ao socket designado. Pensando na conexão com ROS2 é essencial a posição exata da câmera, sendo utilizadas as funções GetSocketLocation() e GetSocketRotation().

4.4 LIDAR

A implementação do LiDAR em código C++, consistiu em simular vários feixes de luz com um alcance angular e direcional alteráveis (o campo de visão é dividido pelo numero de feixes).Utilizaram-se as funções getsocketlocation() e getsocketrotation() para extrair a posição e orientação do socket LiDAR. Com a função LineTraceSingleByChannel() e estrutura FCollisionQueryParams é possivel tracar os feixes até à distância máxima definida ou até colidirem com um objeto visível que não seja a otter. As distâncias alcançadas por cada feixe são guardadas num array unidimensional.

5 Conectar ao ROS2 via rosbridge

Como já referido na primeira fase do recrutamento o ROS2 comunica internamente via DDS, a ligação ao Unreal pode ser feita via rosbridge. O rosbridge cria um servidor de Websockets ao qual se irá ligar o Unreal Engine e enviar mensagens de texto sob o formato JSON.

Porque decidi usar rosbridge? Poderia ter utilizado outras vias de comunicação -UDP por exemplo.Esta via é mais rápida e aconselhada para transmissão de imagem, no entanto mais complexa- teria de programar toda a serialização e desserialização no Unreal e no ROS2. A vantagem do rosbridge é que realiza a serialização automaticamente apesar de consumir mais recursos computacionais e conferir maior latência. Pelas razões descritas e por também falar de rosbridge na primeira fase do meu recrutamente decidi implementá-lo.

5.1 Linux

O ROS2 é instaldo no linux visto ser o suporte nativo da framework.Arranca-se o servidor rosbridge com o comando: ros2 launch rosbridge server rosbridge websocket launch.xml. Por definição o servidor vincula-se ao TCP 9090 e fica em modo de escuta. Usando o comando ip a (ou ifconfig) descobre-se o endereço IP da maquina virtual na rede local.

5.2 Windows- Unreal Engine

Descarreguei a pasta do plugin Rosintegration para o projeto, este plugin foi desenvolvido para UE4, portanto foi necessário resolver alguns erros de compilação. (O plugin rclUE-via UDP é nativo para UE5, mas pelas razões enunciadas acima não o utilizei).

Em termos de comunicação(criação topicos) sensores/ROS2 optei por realizá-la no graph de blueprints ao invés de em código C++ por motivos de simplificação.Basicamente criei vários actor components para não juntar tudo no graph do blueprint barco. A cada actor component corresponde um sensor, por segurança, no Event Begin Play de cada um introduz um módulo Delay para garantir que os tópicos apenas são criados após a ligação com ROS2 está completamente estabelecida. Em termos de nós de comunicação apenas utilizei o Publish Messagen String (para todos menos a câmera) convertendo a informação dos Sensores para formato string(msg texto simples) previamente. Durante os ciclos do Event Tick é o utilizado o módulo cast to para aceder ao blueprint principal e extrair os valores das variáveis específicas (exceto no LiDAR). Acabei por não implentar a transferência de imagem da câmera para ROS2 por ser mais complexo e não poder ser feita recorrendo apenas ao graph do blueprint.